

IXChange BV

SDR-SLC

SDR-SLC Protocol Suite Architecture

Field	Value
Document	SDR-SLC-PS-1.0
Title	SDR-SLC Protocol Suite Architecture
Version	Protocol 1.0 Release 1.2 (function blocks, codec service, transmit profile)
Status	RELEASE CANDIDATE
Date	September 2026
Author	Ivo van Ling, PA2IX
Organisation	IXChange BV
Licence	Creative Commons CC BY 4.0 — Open Source

Copyright © 2026 IXChange BV. All rights reserved. Published under Creative Commons CC BY 4.0.

Table of Contents

Revision History	4
1. Normative Language	4
2. Philosophy and Design Goals	4
2.1 Why SDR-SLC Exists	4
2.2 The Core Architectural Philosophy: Push DSP to the Host	5
2.3 How This Differs from Server-Centric SDR Architectures	5
2.4 Why This Is Ideal for HAM Radio Experimenters	6
2.5 The Target Hardware Cost	6
2.6 What You Get When It Works	7
3. Scope	7
4. Terminology	7
5. Document Set	8
6. The Slice Model: Wideband Capture, Host-Side Processing	8
6.1 What the Hardware Does	8
6.2 What the Client Does	9
6.3 What This Means for Per-Slice Metering	9
6.4 What This Means for Waterfall and Spectrum Display	9
6.5 The Codec Service	9
6.6 Transmit and the Sample Counter	9
7. System Architecture	10
7.1 Protocol Planes	10
7.2 The Device Model: Envelope and Function Blocks	10
7.3 Startup Dependency Order	10
7.4 System Architecture Diagram	11
8. Network Topology	11
8.1 Local Network Operation	11
8.2 IP Addressing	11
8.3 Multiple Devices and Multiple Clients	11
8.4 Managed Switches and IGMP	11
9. Time Synchronisation	12
9.1 Role of PTPv2	12
9.2 Clock Source Hierarchy	12
9.3 The Sample Counter Is Not a Clock	12
10. I/Q Format and Sample Depth	12
11. Conformance	13
11.1 Envelope — every device	13
11.2 Block rx	13
11.3 Block tx	13
11.4 Block codec	14
11.5 Client Conformance	14
12. Typical Session Walkthrough	14
13. Design Rationale	15
13.1 Why mDNS/DNS-SD?	15
13.2 Why JSON/TCP?	15
13.3 Why VITA-49?	15
13.4 Why RTP for the codec block?	15
13.5 Why function blocks?	16
13.6 Why host-side DSP?	16
13.7 Why 64-bit frequency fields?	16
14. Hardware Reference Design	16
14.1 The Tayloe Detector	16
14.2 Microcontroller	16
14.3 ADC Interface	16
14.4 A Minimal Working Build	16
14.5 Adding Transmit	17
15. Security Considerations	17

16. Forward Compatibility.....17

17. References.....18

 17.1 Normative.....18

 17.2 Informative.....18

Appendix A. Identifier Register.....18

Revision History

Rev	Date	Author	Description
1.0	2026-05-01	I. van Ling	Initial public draft.
1.1	2026-05-23	I. van Ling	Revised for implementers. Added §2 Philosophy and Design Goals with host-offload rationale, HAM builder narrative, hardware cost discussion, and architectural comparison with server-centric SDR systems. Added §6 Slice Model. Four-profile conformance model. Added §14 Hardware Reference Design.
1.2	2026-09-14	I. van Ling	Aligned with SDR-SLC-CP-1.0 Release 1.3, SDR-SLC-VITA-1.0 Release 1.2 and SDR-SLC-DISC-1.0 Release 1.2. Five-document suite including SDR-SLC-RTP-1.0. Envelope and function block model; tiers per block; conformance rebuilt per block. Codec service and the transmit profile added to the architecture. Shared sample clock. Authentication. Reference implementations named. Figures redrawn. Appendix A corrected.
1.2	2026-09-16	I. van Ling	Re-issued alongside the SDR-SLC-CP-1.0 Release 1.3 and SDR-SLC-VITA-1.0 Release 1.2 amendments of the same date (connectors, zero_point, hardware_monitors channels). No content changes; cross-references to SDR-SLC-CP-1.0 verified against the amended section numbering.

1. Normative Language

The key words SHALL, SHALL NOT, SHOULD, SHOULD NOT, and MAY are to be interpreted as described in RFC 2119.

Keyword	Interpretation
SHALL	Absolute requirement. Non-conformant if omitted.
SHALL NOT	Absolute prohibition.
SHOULD	Strong recommendation. Deviation requires justification.
MAY	Optional. Implementations may include or omit freely.

2. Philosophy and Design Goals

2.1 Why SDR-SLC Exists

Software-defined radio has reached a level of technical maturity where the barriers are no longer theoretical. The theory is well understood, the algorithms are published, the hardware components are cheap. The remaining barriers are practical: cost, platform exclusivity, and closed interfaces. SDR-SLC is a direct response to all three.

Modern networked SDR transceivers represent a genuine architectural advance: I/Q samples flow over Ethernet, signal processing runs on a connected computer, and the hardware itself is kept relatively simple. The approach is sound and has proven itself in professional and serious-amateur use. The problem is not the architecture. The problem is what surrounds it in the products that implement it.

Cost is the first barrier. High-end SDR transceivers from established manufacturers are priced for the professional and the serious contestator. The experimenter on a modest budget, the student learning the craft, the HAM operator who wants to understand what is inside their radio rather than just use it — these people are priced out of the market before they start.

Platform exclusivity is the second. The client software that accompanies most high-end networked SDR hardware is written for Windows. An engineer who has spent a career in telecommunications and has used Apple hardware for two decades should not be required to maintain a Windows installation to operate a radio.

Proprietary interfaces are the third. Control protocols and streaming formats are frequently documented only to the extent required for third-party plugin authors, if at all. Implementing a fully conformant client from first principles — without access to a reference implementation or a complete specification — is impractical for the individual experimenter.

NOTE	SDR-SLC (Super Low Cost / Slice) is a response to these three constraints. The dual name is intentional: the hardware target is Super Low Cost by design, and the software model is built around independently tunable slices within a wide captured bandwidth.
-------------	---

2.2 The Core Architectural Philosophy: Push DSP to the Host

The fundamental architectural decision in SDR-SLC is the most important one, and it is worth stating clearly: the radio hardware does as little as possible. The host computer does everything else.

The radio hardware has three responsibilities: capture I/Q samples from the antenna, convert them to digital form, and stream them over Ethernet. That is it. It does not demodulate. It does not filter. It does not compute FFTs. It does not decide which frequency you are listening to within the captured bandwidth. It sends raw wideband I/Q samples and gets out of the way.

Everything that requires computation — the digital downconversion that extracts a specific slice from the wideband capture, the filtering, the demodulation, the FFT that drives the waterfall, noise blanking, AGC, squelch, digital voice framing — runs on the host computer. On a MacBook with Apple Silicon, this is trivial.

This is not a compromise. It is a deliberate choice that produces a better system in every dimension that matters for the target use case:

- The hardware is simpler, cheaper, and easier to build. An ADC, a microcontroller, and an Ethernet interface are all you need. No FPGA. No DSP chip.
- The DSP is more capable, not less. A MacBook runs more compute than any embedded FPGA that would fit in a €100 budget.
- The DSP is completely software-defined in the truest sense. Adding a new digital mode is a client software change.
- The S-meter and other per-slice measurements are computed in the client from the slice I/Q, where all the relevant context is known.

There is one deliberate exception to "everything on the host", and it exists for legal rather than technical reasons. The vocoders used by the digital voice modes in general amateur use are patent-encumbered and cannot be embedded in an open client. SDR-SLC therefore defines a codec service: a small network device, discovered and controlled exactly like a radio, that converts between framed digital voice and audio using whatever hardware or software the operator is licensed to run. The client does everything it lawfully can — detection, framing, display — and delegates only the vocoder. See §6.5.

2.3 How This Differs from Server-Centric SDR Architectures

Some networked SDR systems place the signal processing on the radio hardware. The radio contains a powerful FPGA or DSP that performs digital downconversion, channelisation, and spectrum analysis internally. The client receives pre-processed slices and pre-computed FFT data.

This is a legitimate architectural choice and it works well for its target use case. It enables thin clients and lower network bandwidth. But the signal processing FPGA drives up the hardware price dramatically, and it fixes the DSP algorithms in hardware. Adding a new digital mode requires a firmware update from the manufacturer.

SDR-SLC inverts this assumption. The radio is deliberately dumb. The client is deliberately smart.

Concern	Server-centric SDR	SDR-SLC
Signal processing location	Radio hardware (FPGA/DSP)	Host computer (CPU/GPU)
Per-slice metering	On the radio	On the client
Waterfall/FFT	Pre-computed on radio, streamed	Computed on host from wideband I/Q
Hardware cost	High (FPGA required)	Low (ADC + MCU + Ethernet)
DSP flexibility	Fixed by firmware	Fully programmable in client software
Platform requirements	Often Windows-centric	macOS primary, platform-agnostic protocol
New digital modes	Firmware update required	Client software update only
Vocoders	On the radio	Separate codec service, or none
Network data rate	Low (narrow slice I/Q)	Higher (wideband I/Q)

Concern	Server-centric SDR	SDR-SLC
Home builder feasibility	No (FPGA expertise required)	Yes (STM32 + Tayloe detector)

NOTE	The higher network data rate is not a problem in practice. A 2 MHz wideband stream at 16-bit I/Q requires approximately 64 Mbit/s. A standard home Ethernet switch handles this trivially.
-------------	--

2.4 Why This Is Ideal for HAM Radio Experimenters

The HAM radio community has a long tradition of home construction. SDR-SLC is designed to support this tradition at the intersection of modern RF technology and software-defined radio.

The hardware side is genuinely buildable by a competent HAM with basic electronics skills. The Tayloe detector — the quadrature sampling detector used in the IXChange reference design — is an elegantly simple circuit: four bilateral switches, two op-amps, and a clock source. It was invented by Dan Tayloe (N7VE) and described in QEX magazine. The I/Q output feeds directly into the ADC inputs on the microcontroller.

The microcontroller side is equally accessible. The STM32H7 family is well supported by open-source toolchains, has a large and active community, and costs under €15 in single quantities. Every peripheral needed for SDR-SLC is built into the chip. The lwIP TCP/IP stack is free and runs comfortably on the H7.

The software side on the host is taken care of. The SDR-SLC client runs on macOS and handles all the DSP. The HAM builder's job is the hardware and the firmware. A working SDR receiver is achievable in a weekend of building and a week of firmware development.

TIP	You do not need to build the IXChange reference design to get started. An RTL-SDR dongle (€25) bridged to SDR-SLC via <code>sdr-slc-rtld</code> on a Raspberry Pi (€40) is a fully conformant rx Tier 2 device. A Nuand bladeRF 2.0 micro with <code>sdr-slc-bladerfd</code> is a conformant transceiver with scheduled-burst transmit. Both daemons are open source and are the reference implementations named throughout the suite. Start with one of them, learn the protocol, then build your own hardware when you are ready.
------------	---

2.5 The Target Hardware Cost

A complete SDR-SLC RX-only implementation, from antenna connector to Ethernet port, should cost under €50 in components. A full transceiver implementation including a simple PA for QRP operation should cost under €100.

Component	Example	Approx. cost
Microcontroller	STM32H743 or similar H7 series	€10–15
Ethernet PHY	LAN8720A or integrated	€3–5
Quadrature clock source	SI5351A (I2C synthesiser)	€3–5
Tayloe detector switches	74HC4066 or FST3253	€2–3
I/Q buffer amplifiers	NE5532 or OPA2134	€2–3
Band-pass / low-pass filters	LC filter, plug-in modules	€5–10
PCB (5 boards)	2-layer, ~10 cm × 8 cm	€5–10
Passives, connectors, power regulation	Various	€5–10
Total (RX only)		~€35–60
PA module for TX (QRP, 5 W)	IRF510 or similar	€10–20
T/R relay and driver	Reed relay or solid state	€5–10
Total (RX/TX transceiver)		€50–90

WARNING	The firmware is not trivial. Streaming I/Q samples at 1–2 MSPS continuously over Ethernet, assembling VITA-49 packets, managing TCP and UDP simultaneously, and computing fractional timestamps requires careful firmware design. Allow several weekends for the firmware if you are new to STM32 development. But it is achievable, and there are two reference daemons to read.
----------------	---

2.6 What You Get When It Works

When a working SDR-SLC device is connected to the network and the client is running, the experience is that of a high-quality software-defined receiver or transceiver, controlled from macOS with a modern native interface. The waterfall shows the full captured bandwidth. Multiple simultaneous receive slices can be positioned anywhere in it, each demodulating independently with its own mode, passband, and audio output.

For a transceiver, pushing PTT switches the radio to transmit. The client generates the I/Q baseband, packages it as VITA-49 UDP packets, and sends it to the radio hardware. For digital voice modes with slot structure, the client asks the radio for its sample counter and schedules each burst on an exact sample, so a DMR transmission lands inside its 30 ms slot without a millisecond-accurate host clock ever being involved.

For the builder, the gratification is that the complicated part is running on your MacBook where you can inspect it, modify it, and improve it. The hardware is the part you built. The software is open. The protocol is documented. Nothing is a black box.

3. Scope

This document is the primary architectural reference for the SDR-SLC protocol suite. It specifies:

- The design philosophy and architectural goals of the suite (§2).
- The four protocol planes (SLC-DISC, SLC-CP, VITA-SLC, SLC-RTP) and their roles.
- The device model: envelope and function blocks.
- The relationships and dependencies between protocols.
- Network topology, IP addressing, and multicast requirements.
- The I/Q stream and slice model, and the codec service model.
- The role of PTPv2 time synchronisation and of the device sample counter.
- Conformance requirements for devices and clients, per function block.
- Hardware reference design guidance.

NOTE

Protocol-level command details, packet formats, and field definitions are in the individual specifications. Read this document first for context, then read the sub-specifications for implementation details.

4. Terminology

Term	Definition
SDR-SLC Client	Host software on macOS, Linux, or Windows that connects to SDR-SLC devices and performs all signal processing.
SDR-SLC Device	Any hardware or software that acts as an SLC-CP server. A radio, a codec service, or both.
Envelope	The part of SLC-CP every device implements: framing, hello, get_capabilities, error codes, forward compatibility.
Function block	A named, independently claimable set of capabilities and commands: rx, tx, codec. A device implements the envelope plus any combination.
Tier	A build target within one function block. rx has Tiers 1–3, tx Tiers 1–3, codec Tiers 1–2. A device may be at different tiers per block.
SLC-DISC	Service discovery layer. mDNS/DNS-SD advertisement. How clients find devices on the LAN.
SLC-CP	Control plane. JSON/TCP protocol for configuration and stream session management.
VITA-SLC	I/Q stream transport. VITA-49 constrained profiles for UDP sample streaming in both directions.
SLC-RTP	Framed media transport. RTP payload formats for the codec function block.
VITA-A/1.0	The receive IF Data packet profile. Defined in SDR-SLC-VITA-1.0.
VITA-A/1.0-CTX	The Context packet profile. Carries signal type, encoding and Reference Level.
VITA-A/1.0-TX	The transmit IF Data packet profile: sample-count timestamp and burst delimiters.
Slice	A client-side receiver defined by a centre frequency and passband width, derived from the wideband I/Q stream by DDC. There is no hardware concept of a slice.

Term	Definition
Wideband stream	The full captured bandwidth from the hardware ADC, streamed to the client.
DDC	Digital Downconverter. Client-side frequency translation, low-pass filter and decimator.
Codec service	An SDR-SLC device implementing the codec block: converts between audio and framed digital voice over RTP.
Conversion	One codec instance: one input format, one output format, one direction.
Function	The SLC-CP parameter naming which block a command addresses. Replaced <code>direction</code> in CP Release 1.2.
Channel	One independently tunable signal path within a block, on hardware that has several.
Sample counter	The device's free-running count of converted samples, per direction. Read with <code>get_timestamps</code> ; the domain of transmit burst scheduling.
Stream ID	Opaque identifier assigned by the device to a stream session. Distinct from the numeric VITA-49 Stream Identifier.
Reference Level	Power at the antenna port corresponding to a full-scale I/Q sample, in dBm.
OUI	Organizationally Unique Identifier. IXChange BV: 0x402814 (40-28-14).
Taylor detector	A quadrature sampling detector invented by Dan Taylor (N7VE).
mDNS / DNS-SD	RFC 6762 / RFC 6763. Used by SLC-DISC.

5. Document Set

The SDR-SLC Protocol Suite consists of five documents. This document is the primary architectural reference. Read it first.

Document ID	Title	Purpose	Read order
SDR-SLC-PS-1.0 Release 1.2	Protocol Suite Architecture	Philosophy, architecture, device model, conformance summary, hardware guidance.	1 — read first
SDR-SLC-DISC-1.0 Release 1.2	SLC-DISC Service Discovery	mDNS/DNS-SD advertisement, two service types, TXT record fields, IGMP, verification.	2
SDR-SLC-CP-1.0 Release 1.3	SLC-CP Control Plane	JSON/TCP control protocol. Envelope, function blocks, all commands, authentication, conformance.	3
SDR-SLC-VITA-1.0 Release 1.2	VITA-SLC I/Q Stream Transport	VITA-49 receive, context and transmit profiles, timestamps, Reference Level.	4
SDR-SLC-RTP-1.0	SLC-RTP Framed Media Transport	RTP payload formats and packetisation for the codec block.	5, if building or using a codec service

Release numbers are document revisions. The protocol version negotiated on the wire is 1.0 in every case, and is the only version defined.

6. The Slice Model: Wideband Capture, Host-Side Processing

The slice model is the architectural concept that distinguishes SDR-SLC from hardware-centric SDR systems and that justifies the low hardware cost.

6.1 What the Hardware Does

The SDR-SLC hardware captures a wide swath of spectrum from the antenna. With a 2 MSPS ADC, it captures 2 MHz of spectrum centred on whatever frequency the local oscillator is tuned to. This captured signal is converted to digital I/Q samples and streamed over Ethernet to the host. Every sample of every frequency within that band is present in the stream.

The hardware has no knowledge of what is in that spectrum. It does not know you are listening to an SSB station at 14.205 MHz. It simply captures everything and streams it.

6.2 What the Client Does

The client receives the wideband I/Q stream and creates software-defined slices within it. To listen to a USB station at 14.205 MHz when the hardware is centred on 14.1 MHz, the client creates a slice centred 105 kHz above the hardware LO. The DDC downconverts that part of the spectrum to baseband, applies a low-pass filter with the appropriate passband, and passes the result to the demodulator.

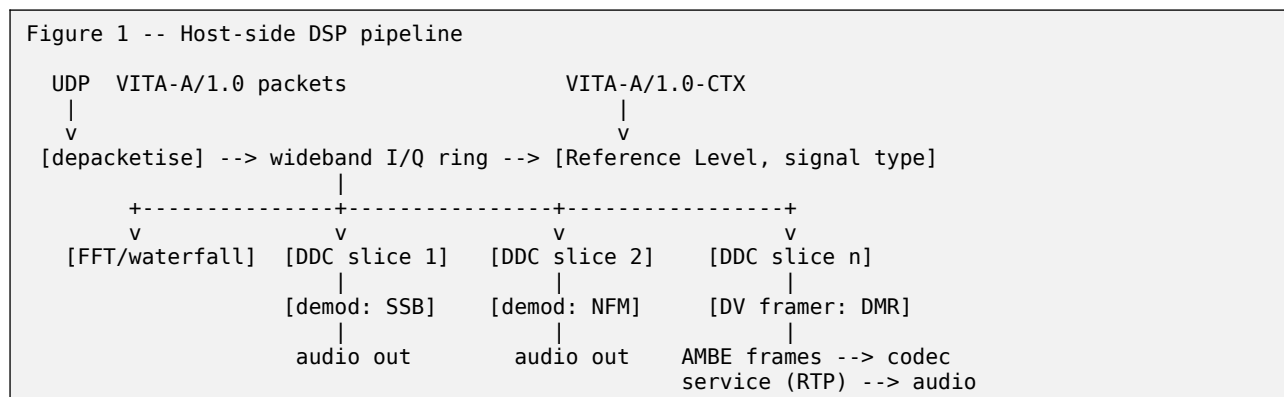
The same wideband stream can simultaneously support multiple slices. The hardware does not know or care.

6.3 What This Means for Per-Slice Metering

Because slices are entirely client-side, the S-meter for each slice is also computed client-side: after DDC and filtering, the RMS amplitude of the decimated slice I/Q gives the signal strength in dBFS. Combined with the Reference Level from the VITA-49 Context packet, this converts to absolute power in dBm at the antenna port. The measurement is made after the passband filter, which is physically correct for an S-meter.

6.4 What This Means for Waterfall and Spectrum Display

The waterfall and spectrum display are computed from the full wideband stream using an FFT. The hardware sends no FFT data. A single ADC output feeds both the spectrum display and all the slice demodulators simultaneously.



6.5 The Codec Service

Digital voice modes (DMR, D-STAR, YSF) carry speech through a vocoder. The client can detect the transmission, recover its framing, correct its errors and display its metadata from the I/Q stream alone — that is signal processing and it belongs on the host. What it cannot do is run the vocoder, because the vocoder is patented and an open client cannot ship it.

The codec service is the answer. It is an SDR-SLC device like any other: discovered with mDNS on its own service type, controlled with SLC-CP through the same envelope, and it carries data over RTP rather than VITA-49 because its payload is short frames on a fixed cadence rather than a continuous sample stream. It declares the formats it understands and the conversions it can perform between them; a client opens a conversion, sends frames to the port the service returns, and receives audio at the port it asked for. A conversion the service cannot do in one step is built from two by pointing one instance's output at another's input; the network is the pipeline.

The service may be a vocoder chip on the radio board, a USB dongle on a spare Linux box, or pure software for the unencumbered codecs. The protocol does not care, and the client never needs to know.

6.6 Transmit and the Sample Counter

Transmit is the receive path reversed: the client generates baseband I/Q and sends VITA-49 packets to the device's listen port. For voice and CW that is all that is needed.

TDMA digital voice adds a constraint the host clock cannot meet: a DMR burst must occupy 27.5 ms of a 30 ms slot, timed to the slot structure the client observed on receive. The only clock exact enough is the device's own sample clock. So a Tier 2 transmitter exposes its receive and transmit sample counters over the control plane

(`get_timestamps`), and the transmit packet profile carries the sample on which a burst must start. The client converts a received sample index into a transmit schedule by arithmetic on those counters; no wall clock is involved. This is what `sdr-slc-bladerfd` implements and what the client's DMR transmit uses.

7. System Architecture

7.1 Protocol Planes

An SDR-SLC system has two kinds of endpoint: SDR-SLC Devices and SDR-SLC Clients. The protocol planes operate simultaneously and independently:

Plane	Protocol	Transport	Default port
Discovery	SLC-DISC (mDNS/DNS-SD)	UDP multicast 224.0.0.251	5353
Control	SLC-CP (JSON/TCP)	TCP unicast	4620
Data, I/Q	VITA-SLC (VITA-49)	UDP unicast	4621 device source (rx), 4622 device listen (tx); client side negotiated
Data, framed voice	SLC-RTP (RTP)	UDP unicast	Negotiated per conversion
Time sync	PTPv2 (IEEE 1588)	UDP multicast / L2	319 / 320

7.2 The Device Model: Envelope and Function Blocks

Every SDR-SLC device consists of an envelope and one or more function blocks. The envelope is what every device shares: discovery, framing, hello, `get_capabilities`, the error registry and the forward-compatibility rules. A function block is a named set of capabilities and commands that a device may or may not implement.

```

+-- ENVELOPE (every device) -----+
| mDNS . TCP 4620 . hello . get_capabilities . errors |
+-----+
| +-- rx -----+ +-- tx -----+ +-- codec -----+ |
| | tune, gain, | | tune, PTT, power | | formats, |
| | stream out | | stream in, | | conversions, |
| | VITA-A/1.0(-CTX) | | bursts | | RTP in/out |
| | Tier 1 / 2 / 3 | | VITA-A/1.0-TX | | Tier 1 / 2 |
| +-----+ +-----+ +-----+ |
| +-----+ |
+-----+

sdr-slc-rtld      : envelope + rx (Tier 2)
sdr-slc-bladerfd : envelope + rx (Tier 2) + tx (Tier 2)
codec service    : envelope + codec (Tier 1 or 2)
STM32 + vocoder  : envelope + rx + tx + codec, advertised as two instances

```

A device declares the blocks it implements as keys in `get_capabilities`; absence of a key is absence of the function. Each block declares its own tier, its own access model (exclusive or shared) and its own authentication requirement. A client reads the capabilities and adapts; it never decides on the tier.

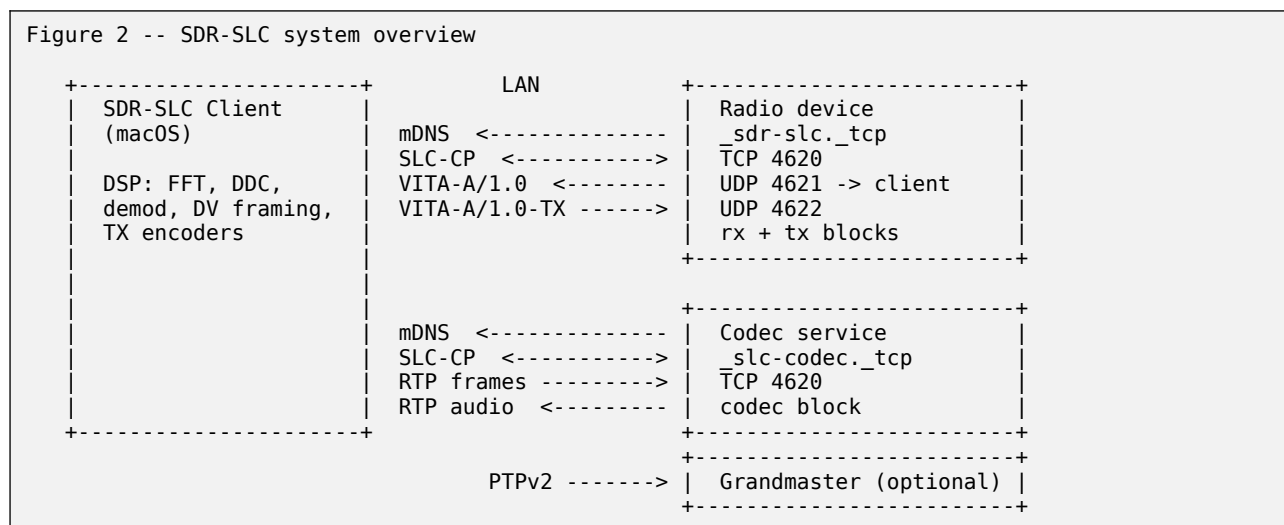
7.3 Startup Dependency Order

The planes have a strict dependency order. The client must complete each phase before beginning the next:

- SLC-DISC first. Browse `_sdr-slc._tcp.local.` for radios and `_slc-codec._tcp.local.` for codec services; resolve hostnames; read TXT records for identity and functions.
- SLC-CP second. Connect TCP, negotiate protocol version, query capabilities, authenticate if a needed block requires it, configure, open stream sessions.
- VITA-SLC or SLC-RTP third. After the start command, UDP packets flow in the negotiated direction.

TIP	PTPv2 operates independently and in parallel. The device listens for PTP SYNC messages from a grandmaster clock and disciplines its internal clock automatically. The client reads clock status via SLC-CP <code>get_clock_status</code> at any time.
------------	---

7.4 System Architecture Diagram



8. Network Topology

8.1 Local Network Operation

All SDR-SLC protocols are designed for operation on a local area network. mDNS is link-local and cannot cross IP routers without special configuration. SLC-CP, VITA-SLC and SLC-RTP use unicast TCP and UDP that can technically cross routers, but latency-sensitive I/Q streaming is strongly recommended on the same LAN segment, and a transmitter SHALL NOT be reachable from the Internet at all (§15).

8.2 IP Addressing

Devices may use static or DHCP-assigned IPv4 addresses. SLC-DISC resolves the device hostname to whatever IP address the device currently holds, so the client is never affected by IP address changes. Every UDP endpoint a device or client listens on is negotiated in `open_stream`; nothing assumes a fixed port on the other side.

8.3 Multiple Devices and Multiple Clients

Multiple SDR-SLC devices on the same LAN are fully supported. Clients may connect to multiple devices simultaneously, each over an independent TCP control connection.

Multiple clients on one device are also supported, and the behaviour is specified rather than left to the implementation. A receive channel is an exclusive resource: one client holds control, others may observe. A codec service is a shared resource: requests are served or refused with busy. Ownership is released on `close_stream` or on loss of the control connection. See SDR-SLC-CP-1.0 §17.

8.4 Managed Switches and IGMP

On managed switches with IGMP snooping, the device must send an IGMPv2 Membership Report for multicast group 224.0.0.251 on Ethernet link-up, before the mDNS announcement. Without this report, the switch will not forward mDNS multicast to the device port and discovery will fail.

NOTE

If your device is not discovered on a managed switch, this is almost always the cause. On an STM32 with lwIP, set `LWIP_IGMP=1` and call `igmp_joingroup()` for 224.0.0.251 after the network interface comes up.

9. Time Synchronisation

9.1 Role of PTPv2

VITA-SLC receive packets carry a UTC timestamp with picosecond fractional resolution. The quality of that timestamp depends on the device's clock source. For most home station use the absolute accuracy of the timestamp does not matter; the fractional timestamp derived from the sample counter is always exact relative to the start of the stream.

Where absolute accuracy matters is frequency calibration, time-domain analysis, and multi-receiver correlation. For these applications, PTPv2 synchronisation to a GPS-disciplined grandmaster provides sub-microsecond UTC accuracy.

9.2 Clock Source Hierarchy

Source	Accuracy	Notes
GPS PTPv2 grandmaster	< 1 μ s	Best. UTC-traceable.
Non-GPS PTPv2 grandmaster	< 100 μ s	Good for most purposes.
On-device GPS module	< 1 μ s	Good. Requires optional GPS module.
Manual (SLC-CP set_clock)	Client accuracy	Acceptable for casual logging.
Host clock (free_running)	Host NTP accuracy	What both reference daemons do. Stamped as UTC, reported as <code>synced: false</code> . Enough to align two receivers to the millisecond.
No time source	Relative only	Bare microcontroller with no RTC. Marked on the wire with TSI=11b.

9.3 The Sample Counter Is Not a Clock

Transmit burst scheduling (§6.6) does not use any of the above. It uses the device's sample counter, which is exact by construction and needs no discipline. The two mechanisms are independent: a device with no time source at all can still place a DMR burst perfectly, and a PTP-disciplined device still schedules bursts by sample count.

TIP	For a first build, a host clock or no clock is perfectly acceptable. Add PTP support when you need frequency calibration or multi-receiver operation.
------------	---

10. I/Q Format and Sample Depth

The VITA-49 streaming profile supports per-stream format negotiation. The client reads the available formats from `capabilities.rx.iq_formats` (or `capabilities.tx.iq_formats`) and selects the most appropriate one. The format is confirmed by the device in the `open_stream` response and remains fixed for the stream lifetime; the Context packet then describes it in-band.

Label	Encoding	Bits	Bytes/pair	Notes
u8	Unsigned offset binary	8	2	Native on RTL-SDR. Lowest bandwidth.
int16	Two's complement	16	4	Native on most 16-bit ADCs and on the bladeRF after scaling.
int24	Two's complement	24	8	High dynamic range ADCs. Left-justified in 32-bit container.
float32	IEEE 754 single	32	8	Normalised to ADC full scale.

NOTE	All VITA-SLC encodings use big-endian byte order. Samples are interleaved. An MTU-safe packet carries 360 IQ pairs at int16 (1496-byte datagram).
-------------	---

11. Conformance

Conformance is claimed per function block, at a tier of that block. A device implements the envelope plus the blocks it advertises in `functions=` and in `get_capabilities`; each block declares its tier. A device may implement rx at Tier 2 and tx at Tier 1. The tables below summarise; the full lists are in SDR-SLC-CP-1.0 §24, SDR-SLC-VITA-1.0 §12 and SDR-SLC-DISC-1.0 §11.

11.1 Envelope — every device

Requirement	Protocol	Level
mDNS advertisement on the correct service type(s), with <code>functions</code> and <code>fw</code>	SLC-DISC	SHALL
IGMPv2 Membership Report on link-up	SLC-DISC	SHALL
mDNS goodbye on clean shutdown	SLC-DISC	SHOULD
Length-prefixed JSON framing; <code>hello</code> ; <code>get_capabilities</code>	SLC-CP	SHALL
Error codes from the registry only; vendor names prefixed	SLC-CP	SHALL
Stop all streams on control connection loss	SLC-CP	SHALL

11.2 Block rx

Requirement	Protocol	T1	T2	T3
<code>set_frequency</code> , <code>set_sample_rate</code> , <code>set_control(rf_gain)</code>	SLC-CP	SHALL	SHALL	SHALL
<code>open_stream</code> , <code>start_rx</code> , <code>stop_rx</code> , <code>close_stream</code> ; <code>stream_state</code> event	SLC-CP	SHALL	SHALL	SHALL
IF Data packets (VITA-A/1.0) to the client's destination	VITA-SLC	SHALL	SHALL	SHALL
<code>ping</code> , <code>get_status</code> , <code>get_control</code> , <code>get_clock_status</code>	SLC-CP	MAY	SHALL	SHALL
Standard controls (<code>agc</code> , <code>bias_tee</code> , <code>ppm</code> , <code>direct_sampling</code>) as fitted	SLC-CP	MAY	SHALL	SHALL
Reference Level (<code>capabilities</code> + <code>get_status</code>)	SLC-CP	MAY	SHALL	SHALL
Context packets (VITA-A/1.0-CTX), VITA-49.0 encoding	VITA-SLC	MAY	SHALL	SHALL
Full event model	SLC-CP	MAY	SHALL	SHALL
<code>hardware_monitors</code> object	SLC-CP	Omit	SHALL	SHALL
<code>set_clock</code> , <code>get_location</code> , <code>set_location</code> ; GPS or PTP	SLC-CP / PTPv2	MAY	SHOULD	SHALL

11.3 Block tx

Requirement	Protocol	T1	T2	T3
<code>get_tx_capabilities</code> , <code>start_tx</code> , <code>stop_tx</code> ; function: "tx" on configuration commands including <code>set_control</code>	SLC-CP	SHALL	SHALL	SHALL
<code>turnaround_mode</code> , <code>full_duplex</code> , <code>tx_data_timeout_ms</code> in <code>capabilities.tx</code>	SLC-CP	SHALL	SHALL	SHALL
Accept VITA-A/1.0 packets on <code>listen_port</code> as continuous transmission	VITA-SLC	SHALL	SHALL	SHALL
Stop on control loss and on data timeout; bias tee sequencing	SLC-CP	SHALL	SHALL	SHALL
Authentication implemented and on by default for tx	SLC-CP	SHALL	SHALL	SHALL
<code>set_tx_power</code>	SLC-CP	MAY	SHALL	SHALL
<code>get_timestamps</code> ; VITA-A/1.0-TX scheduled bursts; <code>tx_burst_late</code>	SLC-CP / VITA-SLC	MAY	SHALL	SHALL
<code>get_vswr</code> , <code>auto_tune</code> , <code>monitor_alert</code>	SLC-CP	Omit	Omit	SHALL

11.4 Block codec

Requirement	Protocol	T1	T2
formats and conversions in capabilities; open_stream with from/to	SLC-CP	SHALL	SHALL
start_codec, stop_codec, close_stream	SLC-CP	SHALL	SHALL
RTP input port bound and returned with a reachable address; output to the client's destination	SLC-CP / SLC-RTP	SHALL	SHALL
Advertise on _slc-codec._tcp	SLC-DISC	SHALL	SHALL
Several concurrent instances; get_status per instance	SLC-CP	MAY	SHALL

11.5 Client Conformance

Requirement	Level
Browse both service types; never reject an instance for an unknown function	SHALL
Parse minhz and maxhz as 64-bit unsigned integers	SHALL
Send hello first offering only "1.0"; call get_capabilities before any configuration	SHALL
Correlate responses by id; track outstanding requests	SHALL
Treat absence of a capability key as absence of the function	SHALL
Render controls[] of every block the device declares, addressed to that block	SHALL
Silently ignore unknown JSON fields, events, blocks, formats	SHALL
Handle unknown_command and not_supported by disabling the UI element; stop sending a command that returned unknown_command	SHALL
Not issue set_sample_rate to a block whose clock group has an active stream	SHALL
Authenticate before addressing a block whose auth is not none; prompt on unauthorized	SHALL
Inhibit start_tx while monitor_alert reports tx_inhibited	SHALL
Decode Context packets per VITA-49.0; treat them as authoritative over control-plane fields	SHALL / SHOULD
Use actual_sample_rate_sps for the frequency axis; the nominal rate for exact ratios	SHALL
Validate the UDP source address of received packets	SHOULD
Re-execute the full startup sequence after TCP disconnection	SHALL

12. Typical Session Walkthrough

Figure 3 -- Complete RX session from device boot to stream active

```

DEVICE BOOT
  Daemon starts: Ethernet up, IGMP join 224.0.0.251
  mDNS announcement: SLC-5A5353._sdr-slc._tcp.local. port 4620
  Clock: host NTP -> free_running, synced:false

DISCOVERY (client)
  Browses _sdr-slc._tcp.local. and _slc-codec._tcp.local.
  Resolves SLC-5A5353.local. -> 192.168.1.181
  Reads TXT: proto=sdr-slc fw=sdr-slc-rtld/1.0 functions=rx
             minhz=100000 maxhz=1766000000

CONTROL HANDSHAKE
  C->S hello(["1.0"])
  S->C device="SLC-5A5353", selected=1.0, firmware_version="sdr-slc-rtld/1.0"
  C->S get_capabilities
  S->C rx:{tier:2, preferred rate, preferred gain, Reference Level,
          controls, transports, iq_formats} (no tx, no codec keys)
  C->S get_clock_status -> source=free_running, synced=false -> warn

CONFIGURATION
  C->S set_frequency(rx, 14100000)
  C->S set_sample_rate(rx, 2048000) // preferred_sample_rate_sps

```

```

C->S set_control(rx, rf_gain, 20.0)           // preferred_gain_db

STREAM SETUP
C->S open_stream(rx, vita49, vita:{format u8, destination 192.168.1.20:60000})
S->C stream_id=rx-1, signal_type=complex_iq,
    vita:{source 192.168.1.181:4621, stream_id 1, samples_per_packet 360, format u8}
C->S start_rx(rx-1)
S->C ok + event stream_state(rx-1, streaming=true, function=rx, reason=client_request)

ACTIVE RECEPTION
[UDP] Context packet: CIF0 0x81008000, RL=-30.0 dBm, u8 complex
[UDP] IF Data packets: 360 IQ pairs, ~5689 pkt/s at 2.048 MSPS
Client: FFT -> waterfall; DDC -> slice demodulation -> audio
Client: S-meter from slice I/Q amplitude + Reference Level
Client: get_status every 2 s -> actual_sample_rate_sps for the frequency axis

SHUTDOWN
C->S stop_rx(rx-1)
C->S close_stream(rx-1)
TCP close

```

Figure 4 -- Transmit with scheduled bursts (sdr-slc-bladerfd, DMR)

```

... after the RX session above, on a device with tx ...
S->C hello carries auth:{methods:[psk-hmac-sha256], nonce}
C->S authenticate(psk-hmac-sha256, key_id, HMAC(nonce)) -> functions:[tx]
C->S set_sample_rate(tx, 1024000)           // before any stream is open: shared clock
C->S open_stream(tx, vita49, vita:{protocol_version VITA-A/1.0-TX, format int16})
S->C stream_id=tx-1, vita:{listen_port 4622, stream_id 2}
C->S get_timestamps -> rx_sample_count, tx_sample_count, rx_stream_epoch_sample
C->S start_tx(tx-1)
[UDP] client -> 4622: VITA-A/1.0-TX packets, BURST_START at tx sample N,
    continuation packets, BURST_END; PA gated per burst
C->S stop_tx(tx-1) ; close_stream(tx-1)

```

13. Design Rationale

13.1 Why mDNS/DNS-SD?

mDNS is natively supported on macOS (Bonjour), Linux (Avahi), and Windows 10+. No server infrastructure, no configuration, no IP addresses to remember. DNS-SD TXT records carry structured metadata — functions, frequency range, implementation and version — readable before the TCP connection is opened.

13.2 Why JSON/TCP?

JSON is human-readable and debuggable with standard tools. You can test every command by hand before writing a single line of client code. TCP provides ordered, reliable delivery for a stateful control protocol. Length-prefixed framing is trivial to implement on STM32 with lwIP. Binary protocols would save a few hundred bytes per message at the cost of debuggability — the wrong trade-off for an open community project.

13.3 Why VITA-49?

VITA-49 is an open, documented standard for I/Q transport with built-in timestamping, stream identification, and in-band metadata via Context packets. Using an established standard means the packet format is documented independently of this project and existing VITA-49 analysis tools work on SDR-SLC streams. That claim is only true if the subset really is a subset; VITA-1.0 Release 1.2 corrected the Context packet so that it is.

13.4 Why RTP for the codec block?

RTP is a twelve-byte header with sequence numbers and timestamps, designed for exactly this kind of short framed payload on a fixed cadence, and it has a large installed base of tools. The negotiation that SIP would carry in SDP is carried in JSON over SLC-CP instead, so the codec service needs no SDP parser and no offer/answer state machine.

13.5 Why function blocks?

Because the first non-radio device — a codec service — could not claim any profile of a specification that assumed every device had a frequency. Separating what every device shares (the envelope) from what a receiver, transmitter or codec specifically does lets each be built, tiered, owned and authenticated independently, and lets a fourth kind of device appear without a second protocol.

13.6 Why host-side DSP?

Because the host is incomparably more powerful than any hardware that fits in a €100 budget, and it is already there. Putting DSP on the host also means algorithms are in software. The experimenter has full access to the DSP.

13.7 Why 64-bit frequency fields?

32-bit unsigned integers represent frequencies up to 4.295 GHz. Microwave SDR hardware operates above this limit. Using 64-bit integers in JSON costs nothing on the wire.

14. Hardware Reference Design

This section provides guidance for HAM operators building their own SDR-SLC hardware. It is informative, not normative. Any hardware that correctly implements the protocol is a conformant SDR-SLC device regardless of its internal construction.

14.1 The Tayloe Detector

The quadrature sampling detector invented by Dan Tayloe (N7VE) is the preferred RF front end for SDR-SLC homebrew hardware. It consists of four bilateral CMOS switches (e.g. FST3253, 74HC4066) driven by a quadrature clock at four times the desired receive frequency, feeding two differential integrators. The output is a direct I/Q baseband signal at a fraction of the component count of a conventional superhet front end.

The Tayloe detector has excellent dynamic range for HF use, typically 100+ dB if the op-amp stage is carefully chosen. The SI5351A clock synthesiser, controlled over I2C from the microcontroller, provides the quadrature drive clock, covering the full HF range from 1.8 MHz to 30 MHz.

TIP

The FST3253 is preferred over the 74HC4066 for the bilateral switches. The NE5532 is a cost-effective op-amp choice; the OPA2134 gives a noticeably lower noise floor for weak-signal work.

14.2 Microcontroller

The STM32H743 or H745 is the recommended microcontroller. The H7 series includes a hardware Ethernet MAC, DMA, hardware SPI for the ADC interface, a hardware timer for packet pacing, and sufficient SRAM for lwIP buffers and VITA-49 packet assembly. Lower-cost alternatives include the STM32F407 with an SPI Ethernet module, and the STM32H563.

For a Linux-based implementation, a Raspberry Pi 4 or 5 running `sdr-slc-rtld` with an RTL-SDR dongle is a conformant rx Tier 2 device without any custom hardware, and any x86 or ARM Linux box running `sdr-slc-bladerfd` with a bladeRF 2.0 micro is a conformant transceiver.

14.3 ADC Interface

The STM32H7 has a built-in 16-bit ADC whose free-running rate comes with caveats around input impedance and noise. For best performance, an external ADC with SPI output driven from a precise clock gives better SNR and cleaner sample timing. For a first build, the internal ADC in dual interleaved mode can achieve 1 MSPS with acceptable performance. Start with the internal ADC to validate the firmware.

14.4 A Minimal Working Build

The minimum for a working rx Tier 1 device:

- STM32H743 NUCLEO-H743ZI development board (€25–30). Includes Ethernet connector and USB for programming.

- SI5351A breakout board (€5). Provides the quadrature clock via I2C.
- 74HC4066 or FST3253 (€1–2). Tayloe detector switches.
- 2× NE5532 dual op-amp (€1–2). I and Q differential amplifiers.
- Band-pass filter for the desired band (€5–10).
- Passives from the junk box; SMA connectors.

Total cost for a NUCLEO-based prototype: approximately €35–45. This runs the full SLC-CP envelope and rx block and streams I/Q over Ethernet to the client.

14.5 Adding Transmit

A transmitter adds a DAC path, a PA and T/R switching, and on the protocol side the tx block at Tier 1: PTT, a listen port, and the two safety rules — stop on control loss, stop on data timeout — that make an unattended networked transmitter acceptable. Authentication for the tx block is required by CP Release 1.3 and is a few hundred lines of HMAC-SHA256, which the H7 has in hardware. Scheduled bursts (Tier 2) need a sample counter the firmware can read atomically; plan for that in the DMA design if TDMA modes are a goal.

15. Security Considerations

SDR-SLC Protocol 1.0 is designed for trusted LAN operation. There is no encryption. Authentication is defined for the control plane (SDR-SLC-CP-1.0 §26): a challenge-response over a pre-shared key, declared per function block. A receiver has no particular reason to use it. A transmitter SHALL implement it and SHALL require it by default, because in most jurisdictions the licence holder is answerable for everything radiated from the station, whoever pressed the button.

WARNING	Authentication protects a station against the household and the guest network. It does not encrypt anything and does not make a transmitter safe on a public address. A device with a tx block SHALL NOT be exposed directly to the Internet, with or without authentication; remote access goes through a VPN. The control port of every device MUST be firewalled from the Internet.
----------------	--

Ownership (§8.3) is not a security mechanism; it stops cooperating clients from confusing each other and nothing else. mDNS advertisements are unauthenticated; clients SHOULD verify the proto and manufacturer TXT fields and display device identity to the operator. UDP data paths accept packets from any source unless filtered; both devices and clients SHOULD filter on the peer address negotiated in open_stream. A future protocol version MAY introduce TLS for the control connection.

16. Forward Compatibility

Scenario	Behaviour
Unknown JSON fields	Silently ignored by all receivers.
Unknown SLC-CP commands	Return <code>unknown_command</code> . Connection stays open.
Unknown events, unknown <code>stream_state.reason</code>	Silently ignored by clients.
Unknown function block in capabilities or TXT	Ignored; the instance is still shown.
Unknown VITA-49 packet types	Silently discarded.
Unknown CIFO bits in a Context packet	Skip the corresponding field bytes; process the rest.
Unused Payload Format bits	Ignored; never whole-word matched.
Unknown codec format or conversion	Ignored, together with conversions referring to it.
Unknown authentication method	Client uses one it knows, or reports it cannot authenticate.
New <code>iq_formats</code> entries	Added with <code>supported: false</code> until implemented.
Protocol minor version (1.0 → 1.1)	Backward compatible. Adds optional fields and commands.
Protocol major version (1.x → 2.0)	Breaking. Requires new hello version negotiation.

Document releases are not protocol versions. Until Protocol 1.0 is declared FINAL, a release may change the wire format and says so in its own §1; after that, only a major version may.

17. References

17.1 Normative

Reference	Title	Relevance
SDR-SLC-DISC-1.0 Release 1.2	SLC-DISC Service Discovery	Device discovery; two service types.
SDR-SLC-CP-1.0 Release 1.3	SLC-CP Control Plane	Envelope, function blocks, commands, authentication.
SDR-SLC-VITA-1.0 Release 1.2	VITA-SLC Stream Transport	VITA-49 receive, context and transmit profiles.
SDR-SLC-RTP-1.0	SLC-RTP Framed Media Transport	Codec block transport.
RFC 6762 / RFC 6763	Multicast DNS / DNS-SD	Discovery.
RFC 2119	Key Words for Use in RFCs	Normative keywords.
IEEE 1588-2008	PTPv2	Clock synchronisation.
ANSI/VITA 49.0	VITA Radio Transport Standard	Base VRT packet format.
RFC 3550	RTP	Codec block transport.
RFC 768 / RFC 793	UDP / TCP	Transports.

17.2 Informative

Reference	Title	Notes
IEEE 802 / OUI	OUI Registry	IXChange BV OUI 40-28-14.
RFC 8259	JSON	JSON encoding.
sdr-slc-rtld	Reference implementation	RTL-SDR bridge, rx Tier 2.
sdr-slc-bladerfd	Reference implementation	bladerF 2.0 micro, rx + tx Tier 2, scheduled bursts.
FreeRTOS, lwIP, cJSON	Embedded stacks	Recommended for STM32 implementations.
Tayloe 2001	QEX July/August 2001 — N7VE	Original Tayloe detector description. Required reading for builders.

Appendix A. Identifier Register

Identifier	Type	Value	Description
IXChange BV OUI	IEEE MA-L OUI	40-28-14 (0x402814)	Class Identifier of every SDR-SLC IF Data packet, whoever built the device.
VITA-A/1.0 PCC	VITA-49 PCC	0x0001	Receive IF Data packet class.
VITA-A/1.0-TX PCC	VITA-49 PCC	0x0002	Transmit IF Data packet class.
VITA-A/1.0-CTX	VITA-49 IF Context packet	Type 0x4, no Class Identifier	Context packets carry no Class Identifier; they are associated with their stream by Stream ID.
Context CIFO	VITA-49 CIFO	0x01008000	Reference Level (bit 24) + Payload Format (bit 15); bit 31 on change.
_sdr-slc._tcp	DNS-SD service type	—	Radios: devices with rx or tx.
_slc-codec._tcp	DNS-SD service type	—	Codec services.
SLC-XXYYZZ	mDNS instance name	Per device	XX YY ZZ = last 3 octets of MAC address, uppercase hex. Durable device identity.
TCP 4620	Port	Default	SLC-CP control.

Identifier	Type	Value	Description
UDP 4621 / 4622	Ports	Defaults	Device VITA source (rx) and listen (tx).
1.0	Protocol version	The only one	Negotiated in hello; independent of document releases.